ElGamal Encryption System

Introduction

ElGamal encryption is a **public-key cryptosystem** based on the **Diffie-Hellman key exchange** principle. It was proposed by **Taher ElGamal** in 1985 and provides semantic security against chosen-plaintext attacks. Unlike RSA, ElGamal's security relies on the **discrete logarithm problem**.

Mathematical Foundation

The Discrete Logarithm Problem

Definition: Given a prime number `p`, a generator `g`, and a value `y`, it is computationally hard to find `x` such that:

```
g^x ≡ y (mod p)
```

This difficulty forms the basis of ElGamal's security.

ElGamal Encryption Algorithm

Step 1: Key Generation

The recipient (Bob) generates a public-private key pair:

1. **Choose a large prime number** `p`
2. **Choose a generator** `g` (a primitive root modulo p)
3. **Choose a private key** `x` randomly where `1 < x < p-1`
4. **Compute public key** `y = g^x mod p`

Public Key: `(p, g, y)`
Private Key: `x`

Step 2: Encryption Process

The sender (Alice) wants to send message `M` to Bob:

1. **Obtain Bob's public key**: `(p, g, y)`
2. **Represent message** as an integer `M` where `0 ≤ M < p`
3. **Choose random** `k` where `1 < k < p-1`
4. **Compute two values**:
   - `C1 = g^k mod p`
   - `C2 = M × y^k mod p`
5. **Send ciphertext**: `(C1, C2)`

Step 3: Decryption Process

Bob receives ciphertext `(C1, C2)` and decrypts:

1. **Compute** `s = C1^x mod p`
2. **Compute** `s^(-1) = s^(p-2) mod p` (using Fermat's Little Theorem)
3. **Recover message**: `M = C2 × s^(-1) mod p`

Why this works:

```
s = C1^x = (g^k)^x = g^(kx) mod p
y^k = (g^x)^k = g^(kx) mod p
Therefore: s = y^k

C2 × s^(-1) = (M × y^k) × (y^k)^(-1) = M mod p
```

Numerical Example

Let's encrypt the message "Hello" (we'll use just one character 'H' = 72 for simplicity).

Key Generation

```
1. Choose p = 23 (prime)
2. Choose g = 5 (generator)
3. Choose private key x = 6
4. Compute y = g^x mod p = 5^6 mod 23 = 8

Public Key: (23, 5, 8)
Private Key: 6
```

Encryption (Message M = 19)

```
1. Choose random k = 3
2. Compute C1 = g^k mod p = 5^3 mod 23 = 125 mod 23 = 10
3. Compute C2 = M × y^k mod p = 19 × 8^3 mod 23
   = 19 × 512 mod 23
   = 19 × 6 mod 23
   = 114 mod 23 = 22

Ciphertext: (10, 22)
```

Decryption

```
1. Compute s = C1^x mod p = 10^6 mod 23 = 6
2. Compute s^(-1) = 6^21 mod 23 = 4 (using Fermat's Little Theorem)
3. Recover M = C2 × s^(-1) mod p = 22 × 4 mod 23 = 88 mod 23 = 19 ✓
```

Python Implementation

```python
import random

def mod_exp(base, exp, mod):
    """
    Efficient modular exponentiation using binary method
    Computes (base^exp) mod mod
    """
    result = 1
    base = base % mod
    while exp > 0:
        if exp % 2 == 1:  # If exp is odd
            result = (result * base) % mod
        exp = exp >> 1     # Divide exp by 2
        base = (base * base) % mod
    return result

def mod_inverse(a, p):
    """
    Compute modular inverse using Fermat's Little Theorem
    For prime p: a^(-1) ≡ a^(p-2) (mod p)
    """
    return mod_exp(a, p - 2, p)

class ElGamal:
    def __init__(self, p, g):
        """
        Initialize ElGamal with prime p and generator g
        """
        self.p = p  # Large prime
        self.g = g  # Generator

    def generate_keys(self):
        """
        Generate public and private key pair
        Returns: (public_key, private_key)
        """
        # Choose random private key x
        x = random.randint(2, self.p - 2)

        # Compute public key y = g^x mod p
        y = mod_exp(self.g, x, self.p)

        public_key = (self.p, self.g, y)
        private_key = x

        return public_key, private_key

    def encrypt(self, message, public_key):
        """
        Encrypt a message using public key
```

```
        Args:
            message: integer message (M < p)
            public_key: tuple (p, g, y)
        Returns: ciphertext tuple (C1, C2)
        """
        p, g, y = public_key

        # Choose random k
        k = random.randint(2, p - 2)

        # Compute C1 = g^k mod p
        C1 = mod_exp(g, k, p)

        # Compute C2 = M * y^k mod p
        C2 = (message * mod_exp(y, k, p)) % p

        return (C1, C2)

    def decrypt(self, ciphertext, private_key):
        """
        Decrypt ciphertext using private key
        Args:
            ciphertext: tuple (C1, C2)
            private_key: integer x
        Returns: original message M
        """
        C1, C2 = ciphertext

        # Compute s = C1^x mod p
        s = mod_exp(C1, private_key, self.p)

        # Compute s^(-1) mod p
        s_inv = mod_inverse(s, self.p)

        # Recover message M = C2 * s^(-1) mod p
        message = (C2 * s_inv) % self.p

        return message

# Example Usage
if __name__ == "__main__":
    # Initialize ElGamal with small prime for demonstration
    p = 23
    g = 5
    elgamal = ElGamal(p, g)

    # Generate keys
    public_key, private_key = elgamal.generate_keys()
    print(f"Public Key: {public_key}")
    print(f"Private Key: {private_key}")
```

```
    # Original message
    message = 19
    print(f"\nOriginal Message: {message}")

    # Encrypt
    ciphertext = elgamal.encrypt(message, public_key)
    print(f"Ciphertext: {ciphertext}")

    # Decrypt
    decrypted = elgamal.decrypt(ciphertext, private_key)
    print(f"Decrypted Message: {decrypted}")

    # Verify
    if message == decrypted:
        print("\nEncryption/Decryption Successful!")
```

Key Features of ElGamal

Advantages

1. **Semantic Security**: Same plaintext produces different ciphertexts due to random `k`
2. **Homomorphic Property**: Supports certain mathematical operations on encrypted data
3. **No Patent Restrictions**: Unlike RSA, ElGamal was never patented
4. **Based on Discrete Log**: Different mathematical foundation than RSA

Disadvantages

1. **Ciphertext Expansion**: Ciphertext is twice the size of plaintext
2. **Randomness Required**: Needs secure random number generator for each encryption
3. **Slower than RSA**: More computationally intensive
4. **Key Reuse Issues**: Same `k` value must never be reused

Security Considerations

Important Security Rules

1. **Never reuse `k` value**: If two messages use same `k`, the system is vulnerable
2. **Large prime `p`**: Should be at least 2048 bits for modern security

3. **Random `k` selection**: Must use cryptographically secure random numbers
4. **Generator selection**: `g` should be a primitive root modulo `p`

Attack Vulnerability

If same `k` is used for two messages:

```
C1 remains same
C2_1 / C2_2 = M1 / M2 (reveals ratio of messages)
```

RSA Cryptosystem

Introduction

RSA (Rivest-Shamir-Adleman) is one of the first and most widely used public-key cryptosystems. It was invented in 1977 by **Ron Rivest**, **Adi Shamir**, and **Leonard Adleman** at MIT.

Why RSA is Important?

- **First practical public-key cryptosystem**: Revolutionary breakthrough in cryptography
- **Widely deployed**: Used in SSL/TLS, SSH, PGP, digital signatures
- **Mathematically elegant**: Based on simple yet powerful number theory
- **Asymmetric encryption**: Different keys for encryption and decryption

The Key Problem RSA Solved

Before RSA: Secure communication required sharing secret keys in advance (symmetric cryptography)

After RSA: Public keys can be shared openly, only private keys must be kept secret

---

Mathematical Foundation

RSA security relies on the **difficulty of factoring large numbers**.

Key Mathematical Concepts

1. Prime Numbers

Numbers divisible only by 1 and themselves: 2, 3, 5, 7, 11, 13, ...

2. Modular Arithmetic

```
a ≡ b (mod n)  means  a mod n = b mod n

Example: 17 ≡ 2 (mod 5)  because 17 mod 5 = 2
```

3. Euler's Totient Function φ(n)

φ(n) = count of numbers less than n that are coprime to n

For prime p: `ϕ(p) = p - 1` For two primes p, q: `ϕ(p × q) = (p - 1)(q - 1)`

Example:

```
ϕ(7) = 6  [numbers: 1,2,3,4,5,6 all coprime to 7]
ϕ(15) = ϕ(3 × 5) = (3-1)(5-1) = 2 × 4 = 8
```

4. Modular Multiplicative Inverse

a and **b** are inverses mod n if: `a × b ≡ 1 (mod n)`

Example: `3 × 5 ≡ 1 (mod 7)`, so 3 and 5 are inverses mod 7

5. Euler's Theorem

If gcd(a, n) = 1, then: `a^ϕ(n) ≡ 1 (mod n)`

This is the **mathematical heart of RSA**!

RSA Algorithm - Step by Step

Step 1: Key Generation

The receiver generates public and private key pairs:

```
1. Choose two large prime numbers: p and q
   (Keep these SECRET!)

2. Compute n = p × q
   (n is the modulus, will be PUBLIC)

3. Compute ϕ(n) = (p - 1)(q - 1)
   (Euler's totient, keep SECRET)

4. Choose public exponent e where:
   - 1 < e < ϕ(n)
   - gcd(e, ϕ(n)) = 1
   (e is commonly 65537 for efficiency)

5. Compute private exponent d where:
   - d × e ≡ 1 (mod ϕ(n))
   - d is the modular inverse of e
   (Keep d SECRET!)
```

Public Key: (n, e)
Private Key: (n, d)

Step 2: Encryption

Sender wants to send message **M** to receiver:

```
Given:
- Message M (as integer, 0 ≤ M < n)
- Receiver's public key (n, e)

Compute:
C = M^e mod n

Send ciphertext C to receiver
```

Note: Message must be converted to numerical form first.

Step 3: Decryption

Receiver decrypts ciphertext **C**:

```
Given:
- Ciphertext C
- Private key (n, d)

Compute:
M = C^d mod n

Recover original message M
```

Why RSA Works - Mathematical Proof

Encryption: `C = M^e mod n`

Decryption: `M = C^d mod n`

Substituting:

```
M = C^d mod n
M = (M^e)^d mod n
M = M^(ed) mod n
```

Since `ed ≡ 1 (mod ϕ(n))`, we can write `ed = 1 + k·ϕ(n)` for some integer k

```
M^(ed) = M^(1 + k·ϕ(n))
       = M × M^(k·ϕ(n))
       = M × (M^ϕ(n))^k
       ≡ M × 1^k          [by Euler's Theorem]
       ≡ M (mod n)
```

Therefore, decryption correctly recovers the original message!

Numerical Example (Small Numbers)

Let's walk through RSA with small primes for understanding:

Key Generation

```
Step 1: Choose primes
p = 61
q = 53

Step 2: Compute n
n = p × q = 61 × 53 = 3233

Step 3: Compute ϕ(n)
ϕ(n) = (p-1)(q-1) = 60 × 52 = 3120

Step 4: Choose e
e = 17  [check: gcd(17, 3120) = 1 ✓]

Step 5: Compute d (modular inverse)
d × e ≡ 1 (mod ϕ(n))
d × 17 ≡ 1 (mod 3120)
d = 2753  [verify: 17 × 2753 = 46801 = 15 × 3120 + 1 ✓]

Public Key:  (n=3233, e=17)
Private Key: (n=3233, d=2753)
```

Encryption

Let's encrypt message M = 123

```
C = M^e mod n
C = 123^17 mod 3233

Using fast modular exponentiation:
123^17 mod 3233 = 855

Ciphertext: C = 855
```

Decryption

Decrypt ciphertext C = 855

```
M = C^d mod n
M = 855^2753 mod 3233

Using fast modular exponentiation:
855^2753 mod 3233 = 123

Original message recovered: M = 123
```

Python Implementation

```python
import random
import math

class RSA:
    """
    RSA Cryptosystem Implementation
    """

    def __init__(self):
        self.public_key = None
        self.private_key = None

    def gcd(self, a, b):
        """
        Compute Greatest Common Divisor using Euclidean Algorithm
        """
        while b:
            a, b = b, a % b
        return a

    def extended_gcd(self, a, b):
        """
        Extended Euclidean Algorithm
        Returns (gcd, x, y) where ax + by = gcd
        """
        if b == 0:
            return a, 1, 0
        else:
            gcd, x1, y1 = self.extended_gcd(b, a % b)
            x = y1
            y = x1 - (a // b) * y1
            return gcd, x, y

    def mod_inverse(self, e, phi):
        """
        Compute modular multiplicative inverse of e modulo phi
        Find d such that (e × d) ≡ 1 (mod phi)
        """
        gcd, x, y = self.extended_gcd(e, phi)

        if gcd != 1:
            raise ValueError("Modular inverse does not exist")

        # Make sure d is positive
        return (x % phi + phi) % phi

    def is_prime(self, n, k=5):
        """
        Miller-Rabin Primality Test
        Args:
```

```
            n: number to test
            k: number of rounds (higher = more accurate)
        Returns: True if probably prime, False if composite
        """
        if n < 2:
            return False
        if n == 2 or n == 3:
            return True
        if n % 2 == 0:
            return False

        # Write n-1 as 2^r × d
        r, d = 0, n - 1
        while d % 2 == 0:
            r += 1
            d //= 2

        # Witness loop
        for _ in range(k):
            a = random.randrange(2, n - 1)
            x = pow(a, d, n)

            if x == 1 or x == n - 1:
                continue

            for _ in range(r - 1):
                x = pow(x, 2, n)
                if x == n - 1:
                    break
            else:
                return False

        return True

    def generate_prime(self, bits=16):
        """
        Generate a random prime number with specified bit length
        Note: Using small bits for demonstration; use 1024+ for real security
        """
        while True:
            # Generate random odd number
            num = random.getrandbits(bits)
            num |= (1 << bits - 1) | 1  # Set MSB and LSB to 1

            if self.is_prime(num):
                return num

    def generate_keypair(self, bits=16):
        """
        Generate RSA public and private key pair
        Args:
```

```
        bits: bit length for primes (use 1024+ for real security)
    Returns: ((n, e), (n, d)) - public and private keys
    """
    print(f"\n{'='*60}")
    print(f"GENERATING RSA KEYS ({bits}-bit primes)")
    print(f"{'='*60}")

    # Step 1: Generate two distinct primes
    print("\nStep 1: Generating prime p...")
    p = self.generate_prime(bits)
    print(f"  p = {p}")

    print("\nStep 2: Generating prime q...")
    q = self.generate_prime(bits)
    while q == p:  # Ensure p ≠ q
        q = self.generate_prime(bits)
    print(f"  q = {q}")

    # Step 2: Compute n = p × q
    n = p * q
    print(f"\nStep 3: Computing n = p × q")
    print(f"  n = {n}")

    # Step 3: Compute φ(n)
    phi = (p - 1) * (q - 1)
    print(f"\nStep 4: Computing φ(n) = (p-1)(q-1)")
    print(f"  φ(n) = {phi}")

    # Step 4: Choose e (commonly 65537)
    e = 65537
    # If e doesn't work, find another
    if e >= phi or self.gcd(e, phi) != 1:
        e = 3
        while self.gcd(e, phi) != 1:
            e += 2

    print(f"\nStep 5: Choosing public exponent e")
    print(f"  e = {e}")
    print(f"  gcd(e, φ(n)) = {self.gcd(e, phi)}")

    # Step 5: Compute d (modular inverse of e)
    d = self.mod_inverse(e, phi)
    print(f"\nStep 6: Computing private exponent d")
    print(f"  d = {d}")
    print(f"  Verification: (e × d) mod φ(n) = {(e * d) % phi}")

    # Store keys
    self.public_key = (n, e)
    self.private_key = (n, d)

    print(f"\n{'='*60}")
```

```
        print(f"✓ KEY GENERATION COMPLETE")
        print(f"{'='*60}")
        print(f"Public Key:  (n={n}, e={e})")
        print(f"Private Key: (n={n}, d={d})")

        return self.public_key, self.private_key

    def encrypt(self, message, public_key):
        """
        Encrypt message using RSA public key
        Args:
            message: integer message (M < n)
            public_key: tuple (n, e)
        Returns: ciphertext C
        """
        n, e = public_key

        if message >= n:
            raise ValueError(f"Message {message} must be less than n={n}")

        # C = M^e mod n
        ciphertext = pow(message, e, n)

        return ciphertext

    def decrypt(self, ciphertext, private_key):
        """
        Decrypt ciphertext using RSA private key
        Args:
            ciphertext: encrypted message C
            private_key: tuple (n, d)
        Returns: original message M
        """
        n, d = private_key

        # M = C^d mod n
        message = pow(ciphertext, d, n)

        return message

    def encrypt_string(self, text, public_key):
        """
        Encrypt a text string (character by character)
        """
        n, e = public_key
        encrypted = []

        for char in text:
            m = ord(char)  # Convert to ASCII
            if m >= n:
                raise ValueError(f"Character '{char}' (ASCII {m}) too large for n=
```

```
{n}")
            c = self.encrypt(m, public_key)
            encrypted.append(c)

        return encrypted

    def decrypt_string(self, encrypted, private_key):
        """
        Decrypt a list of encrypted values back to string
        """
        decrypted = []

        for c in encrypted:
            m = self.decrypt(c, private_key)
            decrypted.append(chr(m))

        return ''.join(decrypted)


# Demonstration and Examples
def demonstrate_rsa():
    """
    Comprehensive RSA demonstration
    """
    rsa = RSA()

    # Generate keys with small primes for demonstration
    public_key, private_key = rsa.generate_keypair(bits=16)

    print("\n" + "="*60)
    print("ENCRYPTION DEMONSTRATION")
    print("="*60)

    # Test with single number
    message = 12345
    print(f"\nOriginal Message (numeric): {message}")

    ciphertext = rsa.encrypt(message, public_key)
    print(f"Encrypted Ciphertext: {ciphertext}")

    decrypted = rsa.decrypt(ciphertext, private_key)
    print(f"Decrypted Message: {decrypted}")

    if message == decrypted:
        print("✓ Encryption/Decryption successful!")

    # Test with string (if n is large enough)
    print("\n" + "="*60)
    print("STRING ENCRYPTION DEMONSTRATION")
    print("="*60)
```

```
    try:
        text = "HELLO"
        print(f"\nOriginal Text: '{text}'")

        encrypted_text = rsa.encrypt_string(text, public_key)
        print(f"Encrypted (list of numbers): {encrypted_text}")

        decrypted_text = rsa.decrypt_string(encrypted_text, private_key)
        print(f"Decrypted Text: '{decrypted_text}'")

        if text == decrypted_text:
            print("String encryption/decryption successful!")
    except ValueError as e:
        print(f"Note: {e}")
        print("(Use larger primes for encrypting text)")


# Run demonstration
if __name__ == "__main__":
    demonstrate_rsa()

    # Additional manual example with known small primes
    print("\n" + "="*60)
    print("MANUAL EXAMPLE WITH SMALL PRIMES")
    print("="*60)

    rsa = RSA()

    # Use the example from lecture notes
    p, q = 61, 53
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 17
    d = rsa.mod_inverse(e, phi)

    public_key = (n, e)
    private_key = (n, d)

    print(f"\nManual Setup:")
    print(f"  p = {p}, q = {q}")
    print(f"  n = {n}")
    print(f"  ϕ(n) = {phi}")
    print(f"  e = {e}")
    print(f"  d = {d}")

    # Encrypt and decrypt
    message = 123
    print(f"\nEncrypting message: {message}")
    ciphertext = rsa.encrypt(message, public_key)
    print(f"  Ciphertext: {ciphertext}")
```

```
    decrypted = rsa.decrypt(ciphertext, private_key)
    print(f"  Decrypted: {decrypted}")
    print(f"  Match: {message == decrypted} ✓")
```

Fast Modular Exponentiation

Computing large powers like `M^e mod n` directly is impractical. We use the **binary exponentiation** method:

Algorithm: Square-and-Multiply

```
def fast_mod_exp(base, exponent, modulus):
    """
    Compute (base^exponent) mod modulus efficiently
    Time Complexity: O(log exponent)
    """
    result = 1
    base = base % modulus

    while exponent > 0:
        # If exponent is odd, multiply base with result
        if exponent % 2 == 1:
            result = (result * base) % modulus

        # Square the base
        base = (base * base) % modulus

        # Divide exponent by 2
        exponent = exponent // 2

    return result

# Example: Compute 123^17 mod 3233
result = fast_mod_exp(123, 17, 3233)
print(f"123^17 mod 3233 = {result}")  # Output: 855
```

Example Trace

Computing `5^13 mod 19`:

```
Binary representation of 13: 1101₂

Step  | Exponent | Binary | Action           | Result
------|----------|--------|------------------|--------
Init  | 13       | 1101   | result = 1       | 1
1     | 13       | 1101   | bit=1: r=1×5=5   | 5
      | 6        | 110    | square: 5²=25=6  |
2     | 6        | 110    | bit=0: skip      | 5
      | 3        | 11     | square: 6²=36=17 |
3     | 3        | 11     | bit=1: r=5×17=9  | 9
      | 1        | 1      | square: 17²=4    |
4     | 1        | 1      | bit=1: r=9×4=17  | 17
      | 0        |        | done             |

Final: 5^13 mod 19 = 17
```

RSA Digital Signatures

RSA can also provide **digital signatures** for authentication and non-repudiation.

Signing Process

```
1. Alice has message M
2. Alice computes hash: h = Hash(M)
3. Alice signs with private key: S = h^d mod n
4. Alice sends (M, S) to Bob
```

Verification Process

```
1. Bob receives (M, S)
2. Bob computes hash: h = Hash(M)
3. Bob verifies with Alice's public key: h' = S^e mod n
4. If h = h', signature is valid ✓
```

Why It Works

```
S = h^d mod n               (Alice signs)
h' = S^e = (h^d)^e = h^(de) ≡ h (mod n)  (Bob verifies)
```

Python Implementation

```python
import hashlib

class RSASignature:
    """
    RSA Digital Signature Implementation
    """

    def __init__(self, rsa_instance):
        self.rsa = rsa_instance

    def hash_message(self, message):
        """
        Create hash of message using SHA-256
        Returns integer representation
        """
        if isinstance(message, str):
            message = message.encode()

        hash_obj = hashlib.sha256(message)
        hash_hex = hash_obj.hexdigest()
        hash_int = int(hash_hex, 16)

        return hash_int

    def sign(self, message, private_key):
        """
        Sign a message using private key
        Args:
            message: string or bytes to sign
            private_key: (n, d)
        Returns: signature S
        """
        n, d = private_key

        # Hash the message
        h = self.hash_message(message)

        # Reduce hash to fit within modulus
        h = h % n

        # Sign: S = h^d mod n
        signature = pow(h, d, n)

        return signature

    def verify(self, message, signature, public_key):
        """
        Verify a signature using public key
        Args:
            message: original message
```

```
            signature: S to verify
            public_key: (n, e)
        Returns: True if valid, False otherwise
        """
        n, e = public_key

        # Hash the message
        h = self.hash_message(message)
        h = h % n

        # Verify: h' = S^e mod n
        h_prime = pow(signature, e, n)

        # Check if hashes match
        return h == h_prime


# Demonstration
def demonstrate_signatures():
    """
    Demonstrate RSA digital signatures
    """
    print("\n" + "="*60)
    print("RSA DIGITAL SIGNATURE DEMONSTRATION")
    print("="*60)

    # Setup RSA
    rsa = RSA()
    public_key, private_key = rsa.generate_keypair(bits=512)

    sig_system = RSASignature(rsa)

    # Original message
    message = "This is an important contract."
    print(f"\nOriginal Message: '{message}'")

    # Alice signs the message
    signature = sig_system.sign(message, private_key)
    print(f"\nSignature (Alice's private key): {signature}")

    # Bob verifies the signature
    is_valid = sig_system.verify(message, signature, public_key)
    print(f"\nVerification (Alice's public key): {is_valid}")

    if is_valid:
        print("Signature is VALID - Message is authentic!")

    # Try tampering with message
    print("\n" + "-"*60)
    print("TAMPERING TEST")
    print("-"*60)
```

```
    tampered_message = "This is an important contract!"  # Added '!'
    print(f"\nTampered Message: '{tampered_message}'")

    is_valid_tampered = sig_system.verify(tampered_message, signature, public_key)
    print(f"Verification: {is_valid_tampered}")

    if not is_valid_tampered:
        print("Tampering detected - Signature INVALID!")

# Run signature demonstration
if __name__ == "__main__":
    demonstrate_signatures()
```

Security Considerations

Key Strengths

1. **Mathematically sound**: Based on number theory
2. **Well-studied**: 45+ years of cryptanalysis
3. **Standardized**: PKCS#1, FIPS 186
4. **Versatile**: Encryption, signatures, key exchange

Vulnerabilities & Attacks

1. Small Exponent Attack

Problem: Using e = 3 with small messages

Example:

```
If M^3 < n, then C = M^3 (no modular reduction)
Attacker can compute M = ∛C directly!
```

Solution: Use proper padding (OAEP)

2. Common Modulus Attack

Problem: Same n used with different (e, d) pairs

Solution: Never share modulus between different users

3. Timing Attack

Problem: Measuring decryption time can leak information about d

Solution: Use constant-time implementations

4. Chosen Ciphertext Attack

Problem: Attacker tricks user into decrypting specially crafted ciphertexts

Solution: Use padding schemes (PKCS#1 v1.5, OAEP)

Padding Schemes

Never use textbook RSA in practice! Always use padding:

```
PKCS#1 v1.5:  [0x00] [0x02] [random padding] [0x00] [message]
OAEP:         [mask] [lHash] [PS] [0x01] [message]
```

RSA Key Sizes & Security Levels

Key Size	Security Bits	Status	Year
512 bits	~64-bit	**BROKEN**	1999
768 bits	~80-bit	**BROKEN**	2009
1024 bits	~80-bit	**WEAK**	~2020
2048 bits	~112-bit	**Minimum**	Current
3072 bits	~128-bit	**Recommended**	Current
4096 bits	~140-bit	**High security**	Current

Recommendations (2024)

- **Minimum**: 2048 bits
- **Standard**: 3072 bits
- **High security**: 4096 bits
- **Legacy systems**: Upgrade from 1024 bits immediately!

RSA vs Other Cryptosystems

RSA vs ECC

Feature	RSA	ECC
Key Size (128-bit security)	3072 bits	256 bits
Speed (key generation)	Slow	Fast
Speed (encryption)	Fast (small e)	Moderate
Speed (decryption)	Slow	Moderate
Patent Status	Expired (2000)	Free
Standardization	Widespread	Growing
Quantum Resistance	No	No

RSA vs Symmetric Crypto

Operation	RSA (2048-bit)	AES (256-bit)
Key Size	2048 bits	256 bits
Encryption	~0.1 ms	~0.001 ms
Decryption	~10 ms	~0.001 ms
Use Case	Key exchange, signatures	Bulk data encryption

Hybrid Approach: Use RSA to exchange AES keys, then use AES for data

---

Real-World Applications

1. SSL/TLS (HTTPS)

```
Browser ↔ Server:
1. Server sends RSA public key (in certificate)
2. Browser generates random session key
3. Browser encrypts session key with server's RSA public key
4. Both use session key for AES encryption (faster)
```

2. SSH (Secure Shell)

```
Client authenticates to server using RSA key pair
~/.ssh/id_rsa      (private key)
~/.ssh/id_rsa.pub  (public key)
```

3. PGP/GPG (Email Encryption)

```
- RSA for key exchange
- Symmetric cipher for message encryption
- RSA signature for authentication
```

4. Code Signing

```
Software developers sign code with RSA private key
Users verify authenticity with developer's public key
```

Practical Implementation Considerations

1. Prime Generation

```python
def generate_large_prime(bits=1024):
    """
    Generate cryptographically strong prime
    """
    while True:
        # Generate random odd number
        candidate = random.getrandbits(bits)
        candidate |= (1 << bits - 1) | 1

        # Check divisibility by small primes first
        small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
        if all(candidate % p != 0 for p in small_primes):
            # Run Miller-Rabin primality test
            if miller_rabin(candidate, k=40):
                return candidate
```

2. Random Number Generation

```python
import secrets

# GOOD: Use cryptographically secure random
private_key = secrets.randbelow(phi)

# BAD: Never use standard random for cryptography!
# private_key = random.randint(1, phi)  # ✗ INSECURE!
```

3. Key Storage

```
Private keys should be:
- Encrypted at rest (use AES)
- Protected with strong passwords
- Stored in secure key management systems
- Never hardcoded in source code
- Backed up securely
```

Complete Working Example: Secure Messaging

```
"""
Practical RSA Implementation for Secure Messaging
Demonstrates: Key generation, encryption, decryption, signing, verification
"""

import hashlib
import secrets
import base64

class SecureMessaging:
    """
    Complete secure messaging system using RSA
    """

    def __init__(self):
        self.rsa = RSA()
        self.users = {}  # Store user public keys

    def register_user(self, username, bits=512):
        """
        Register a new user with RSA key pair
        """
        print(f"\nRegistering user: {username}")
        public_key, private_key = self.rsa.generate_keypair(bits)

        # Store public key in directory (like a PKI)
        self.users[username] = {
            'public_key': public_key,
            'private_key': private_key  # Normally user keeps this secret!
        }

        print(f"User {username} registered")
        return public_key, private_key

    def send_secure_message(self, sender, receiver, message):
        """
        Send encrypted and signed message
        """
        print(f"\n{'='*60}")
        print(f"{sender} sending message to {receiver}")
        print(f"{'='*60}")

        # Get keys
        sender_private = self.users[sender]['private_key']
        receiver_public = self.users[receiver]['public_key']

        print(f"\n1. Original Message: '{message}'")

        # Convert message to numbers (simple approach: ASCII values)
        message_numbers = [ord(c) for c in message]
```

```
        print(f"2. As numbers: {message_numbers}")

        # Encrypt each character with receiver's public key
        encrypted = []
        for num in message_numbers:
            if num < receiver_public[0]:  # Check if fits in modulus
                cipher = self.rsa.encrypt(num, receiver_public)
                encrypted.append(cipher)

        print(f"3. Encrypted: {encrypted[:5]}..." if len(encrypted) > 5 else f"3.
Encrypted: {encrypted}")

        # Sign the original message with sender's private key
        sig_system = RSASignature(self.rsa)
        signature = sig_system.sign(message, sender_private)
        print(f"4. Digital Signature: {signature}")

        # Package: (encrypted_message, signature)
        package = (encrypted, signature)
        print(f"\nMessage packaged and ready to send")

        return package

    def receive_secure_message(self, sender, receiver, package):
        """
        Receive, verify, and decrypt message
        """
        print(f"\n{'='*60}")
        print(f"{receiver} receiving message from {sender}")
        print(f"{'='*60}")

        encrypted, signature = package

        # Get keys
        receiver_private = self.users[receiver]['private_key']
        sender_public = self.users[sender]['public_key']

        print(f"\n1. Received encrypted message and signature")

        # Decrypt message with receiver's private key
        decrypted_numbers = []
        for cipher in encrypted:
            num = self.rsa.decrypt(cipher, receiver_private)
            decrypted_numbers.append(num)

        decrypted_message = ''.join([chr(n) for n in decrypted_numbers])
        print(f"2. Decrypted message: '{decrypted_message}'")

        # Verify signature with sender's public key
        sig_system = RSASignature(self.rsa)
        is_valid = sig_system.verify(decrypted_message, signature, sender_public)
```

```
        print(f"3. Signature verification: {is_valid}")

        if is_valid:
            print(f"\nMessage authenticated - Sender is {sender}")
            print(f"Message integrity verified - Not tampered")
            return decrypted_message
        else:
            print(f"\nWARNING: Signature invalid - Message may be forged!")
            return None


# Complete Demonstration
def main_demo():
    """
    Complete demonstration of secure messaging system
    """
    print("="*60)
    print("SECURE MESSAGING SYSTEM DEMONSTRATION")
    print("="*60)

    # Create messaging system
    system = SecureMessaging()

    # Register Alice and Bob
    alice_public, alice_private = system.register_user("Alice", bits=512)
    bob_public, bob_private = system.register_user("Bob", bits=512)

    # Alice sends message to Bob
    message = "Meet at 3pm"
    package = system.send_secure_message("Alice", "Bob", message)

    # Bob receives and verifies
    received = system.receive_secure_message("Alice", "Bob", package)

    print(f"\n{'='*60}")
    print(f"RESULT")
    print(f"{'='*60}")
    print(f"Original:  '{message}'")
    print(f"Received:  '{received}'")
    print(f"Match: {message == received} ")

if __name__ == "__main__":
    main_demo()
```

Common Implementation Mistakes

Mistake 1: Using Textbook RSA

```
# WRONG - No padding
def bad_encrypt(message, public_key):
    n, e = public_key
    return pow(message, e, n)
```

Fix: Always use padding (OAEP)

Mistake 2: Small Exponent with Small Message

```
# WRONG - If M^e < n, no modular reduction occurs
e = 3
M = 100  # Small message
C = M^3  # Can be reversed easily!
```

Fix: Use proper padding or larger e

Mistake 3: Not Validating Inputs

```
# WRONG - No validation
def bad_decrypt(ciphertext, private_key):
    n, d = private_key
    return pow(ciphertext, d, n)
```

Fix: Validate ciphertext range

```
# CORRECT
def good_decrypt(ciphertext, private_key):
    n, d = private_key
    if not (0 <= ciphertext < n):
        raise ValueError("Invalid ciphertext")
    return pow(ciphertext, d, n)
```

Mistake 4: Weak Random Number Generation

```
# WRONG - Not cryptographically secure
import random
key = random.randint(1, n)
```

Fix: Use secrets module

```
# CORRECT
import secrets
key = secrets.randbelow(n)
```

Performance Optimization

Technique 1: Chinese Remainder Theorem (CRT)

For decryption, CRT can speed up computation by ~4x:

```
def decrypt_crt(ciphertext, p, q, d):
    """
    Fast RSA decryption using CRT
    Approximately 4x faster than standard method
    """
    # Compute d_p and d_q
    d_p = d % (p - 1)
    d_q = d % (q - 1)

    # Compute M_p and M_q
    M_p = pow(ciphertext, d_p, p)
    M_q = pow(ciphertext, d_q, q)

    # Compute q_inv
    q_inv = pow(q, -1, p)  # Modular inverse

    # Combine using CRT
    h = (q_inv * (M_p - M_q)) % p
    M = M_q + h * q

    return M
```

Technique 2: Small Public Exponent

```
# Common choice: e = 65537 = 2^16 + 1
# Benefits:
# - Fast encryption (only 17 bits set)
# - Secure (not too small)
# - Fast verification of signatures

e = 65537  # Recommended value
```

Summary & Key Takeaways

Core Concepts

1. **RSA is an asymmetric cryptosystem**
    - Public key (n, e) for encryption
    - Private key (n, d) for decryption
    - Based on factoring difficulty
2. **Mathematical foundation**:

```
Key Generation: n = p×q, ϕ(n) = (p-1)(q-1), ed ≡ 1 (mod ϕ(n))
Encryption:     C = M^e mod n
Decryption:     M = C^d mod n
```

3. **Security relies on**:
    - Difficulty of factoring n into p and q
    - Computational infeasibility of computing φ(n) without knowing factors
    - Proper implementation (padding, key size, random numbers)
4. **Two main uses**:
    - **Encryption**: Confidentiality (hide message content)
    - **Digital Signatures**: Authentication + Non-repudiation
5. **Practical guidelines**:
    - Use **minimum 2048-bit keys** (3072-bit recommended)
    - Always use **padding schemes** (OAEP, PSS)
    - Use **cryptographically secure RNG**
    - Never reuse modulus n between users
    - Combine with symmetric crypto for efficiency